

第5章 内存管理

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 JVM 内存区域的五大划分,并与 C 的栈/堆/静态区对照
- 理解对象在堆上的生命周期与"逃逸"概念
- 理解垃圾回收的基本原理:可达性分析、分代收集、Stop-The-World
- 掌握强/软/弱/虚四种引用,重点会用 WeakReference 与 WeakHashMap
- 识别生产环境四大内存泄漏源,会写防泄漏代码
- 掌握 try-with-resources 资源管理,理解它与 C RAII 的对应关系
- 会用 JVM 参数与诊断工具定位 OOM

💡 从 C 到 Java:内存管理的世界观转变

C 程序员的第一直觉是"内存必须自己管":malloc 配对 free,一个不配就是泄漏。Java 把这套全扔了——new 出来的对象没人负责回收,垃圾回收器(GC)自动处理。第一次写 Java 的人会恐慌:"那我的内存什么时候被释放?会不会到处泄漏?"

答案分两层:**浅层答案**——你不需要管,GC 会找到"再也够不着"的对象并回收,绝大多数内存问题自动消失;**深层答案**——GC 只能回收"不可达"的对象,如果你的代码把对象留着(静态集合、缓存、监听器),GC 永远不会动它,这就是 Java 的内存泄漏。C 的泄漏是"忘了释放",Java 的泄漏是"忘了放手"。

栈与堆的职责也变了。C 里结构体可以在栈上、可以复制;Java 里所有对象一律在堆上,栈上只放"引用"。你写 `Point p = new Point(1,2)`,p 在栈上,Point 对象在堆上——这和 `Point* p = malloc(sizeof(Point))` 同构,只是没有 free。

资源管理需要新工具。C 用 RAII(析构函数)保证文件、锁在退出作用域时释放;Java 没有析构函数,替代方案是 try-with-resources ——写出来像 C 的 RAII,但语法上是显式的 try 块。这是本章的生产力重点。

学习顺序:先看 JVM 内存地图(\$5.1-5.2),再理解 GC 机制(\$5.3-5.4),然后学防泄漏与资源管理(\$5.5-5.6),最后是诊断工具(\$5.7)。

📦 前置知识自查

- C 内存:malloc/free、栈帧、指针、悬垂指针、内存泄漏的概念
- 第1章:数组与字符串(对象本质);第4章:方法调用与栈帧
- 第3章:循环与分支(写实验需要)

🚀 本章学习路线

1. **第一阶段(约 1.5 小时):**\$5.1-5.2 对照 C 的进程内存模型,画一张"JVM 内存地图"
2. **第二阶段(约 1.5 小时):**\$5.3-5.4 GC 原理 + 弱引用,运行 WeakReference 示例观察回收
3. **第三阶段(约 1.5 小时):**\$5.5-5.6 内存泄漏与 try-with-resources,完成章末实验
4. **验收标准:**能解释"Java 内存泄漏=忘了放手";能写出防泄漏的缓存;能写出 try-with-resources

本章知识导图

- §5.1 JVM 内存区域:程序计数器 / 栈 / 堆 / 方法区
- §5.2 栈与堆的生命周期:栈帧 / 对象逃逸
- §5.3 垃圾回收:可达性分析 / 分代收集 / STW
- §5.4 四种引用:强 / 软 / 弱 / 虚引用
- §5.5 内存泄漏四大源头与防护
- §5.6 try-with-resources 资源管理
- §5.7 OOM 诊断与 JVM 参数

§5.1 JVM 内存区域:一张地图

5.1.1 五大区域

JVM 区域	C 的对应物	存什么 / 特点
程序计数器	(无直接对应)	当前执行到哪条字节码;线程私有,极小
虚拟机栈(Java 栈)	调用栈 + 局部变量	每线程一个;栈帧存局部变量、操作数;方法调用→压帧,返回→弹帧
本地方法栈	调用栈	执行 native 方法(如 System.currentTimeMillis 底层)
堆(Heap)	malloc 的堆	所有 new 出来的对象;GC 的主战场;线程共享;最大的一块
方法区/元空间	静态区 (.data/.bss)	类元数据、静态变量、常量池(字符串字面量);JDK8+ 叫元空间,用本地内存

示例 5-1 栈深观察:递归到溢出

```
public class StackDepthDemo {
    static int depth = 0;

    static void dive() {
        depth++;           // 每层递归压一个栈帧
        dive();            // 永不返回:栈帧无限增长
    }

    public static void main(String[] args) {
        try {
            dive();
        } catch (StackOverflowError e) {
            // 注意:Error 不是 Exception,生产代码通常不 catch;这里仅演示
            System.out.println("栈溢出,最大深度约 " + depth);
        }
    }
}
```

// 输出类似:栈溢出,最大深度约 12000(与 -Xss 有关)

⚠ 易错点 1:OOM 与栈溢出是不同的 Error

StackOverflowError:递归过深,栈区耗尽;**OutOfMemoryError**:堆(或元空间)耗尽。两者都是 Error 不是 Exception(第9章详讲)。区分诊断:StackOverflowError 先看递归终止条件;OOM 先看是"Java heap space"还是"Metaspace"还是"unable to create native thread"——三种成因完全不同。

§5.2 栈与堆的生命周期

5.2.1 对象逃逸:栈上的引用,堆上的对象

规则: 局部变量在栈帧里,对象在堆上,栈帧弹出时,引用消失,对象变成"不可达"候选,等待 GC。唯一例外是"局部变量是基本类型"(整个值都在栈上)。"对象被方法返回/存入字段/存入集合,导致它的生命周期超过定义它的方法"——这叫**逃逸**。

示例 5-2 对象生命周期演示

```
public class LifecycleDemo {
    // 堆上对象计数:构造器里 ++,演示"谁还活着"很难靠计数,要靠可达性
    static int alive = 0;

    static class BigData {
        byte[] payload = new byte[1024 * 1024]; // 每个对象占 1MB 堆
        BigData() { alive++; }
        @Override protected void finalize() { alive--; } // 仅演示,生产禁用!
    }

    static BigData make() {
        BigData d = new BigData(); // 对象在堆上,d 是栈上的引用
        return d; // 引用返回:对象逃逸出 make 的栈帧
    }

    public static void main(String[] args) throws Exception {
        System.out.println("初始存活: " + alive);
        BigData keep = make(); // keep 持有引用:对象继续活
        System.out.println("make 后存活: " + alive);

        for (int i = 0; i < 20; i++) {
            make(); // 返回值丢弃:20 个对象立刻不可达
        }
        System.gc(); // 建议 JVM 执行 GC(不保证立即执行)
        Thread.sleep(100); // 给 GC 一点时间
        System.out.println("丢弃后存活: " + alive); // 只剩 keep 那 1 个
    }
}
// 输出趋势:初始 0 → make 后 1 → 丢弃后 1(finalize 已不推荐,结果可能为 2,原理见下)
```

★ 重点:finalize 已废弃,别用

C 的析构函数在 Java 里没有对应物。finalize() 是历史遗留:调用时机不确定、影响 GC 性能, JDK9 起标记废弃,生产代码一律不用。资源释放的正确姿势:try-with-resources(§5.6)。上面示例用 finalize 只为演示"对象消亡"——正常代码统计存活对象应该用分析工具,而不是钩子。

§5.3 垃圾回收:可达性分析

5.3.1 原理:根可达,其余皆垃圾

GC 从 **GC Roots** 出发做可达性分析:栈上的局部变量、静态字段、活跃线程、JNI 引用是根;从根能引用到的对象都"可达、活着";不可达的对象被标记为垃圾,统一回收。这正是"引用计数"方案的替代——C++ 的 shared_ptr 用引用计数,循环引用会泄漏;Java 的可达性分析天然解决循环引用(两个互相引用但外部够不着的对象,一起被回收)。

5.3.2 分代收集:新生代与老年代

- **新生代(Young)**:新对象都先进来;大部分对象"朝生暮死";Minor GC 频繁且快(Eden + 两个 Survivor 区)
- **老年代(Old)**:熬过多次 Minor GC 的对象晋升进来;Major GC 慢、少
- **Stop-The-World(STW)**:GC 线程要暂停业务线程才能安全移动对象;停顿时间 = GC 调优的核心指标

生产结论: 别迷信"自己调 GC 参数",默认 GC(JDK17 默认 G1)对绝大多数应用已足够;你需要做的只是:别把对象留在老年代(防泄漏)、别创建海量短命大对象(给 GC 添乱)。

示例 5-3 观察 GC:内存曲线

```
public class GcDemo {
    public static void main(String[] args) throws Exception {
        Runtime rt = Runtime.getRuntime();
        long used = (rt.totalMemory() - rt.freeMemory()) / 1024 / 1024;
        System.out.println("JVM 总内存(约): " + rt.totalMemory() / 1024 / 1024 + " MB");
        System.out.println("当前已用(约): " + used + " MB");

        for (int round = 0; round < 3; round++) {
            for (int i = 0; i < 1000; i++) {
                byte[] junk = new byte[1024 * 100];    // 每个 100KB,共约 100MB
            }
            System.gc();          // 建议回收(生产不要主动调,这里用于教学观察)
            Thread.sleep(200);
            used = (rt.totalMemory() - rt.freeMemory()) / 1024 / 1024;
            System.out.println("第 " + (round + 1) + " 轮后已用(约): " + used + " MB");
        }
        // 观察点:每轮创建 100MB 垃圾后,已用内存没有累积到 300MB—GC 把它们回收了
    }
}
```

⚠ 易错点 2:System.gc() 只是"建议"

生产代码 不要调 System.gc():它只是向 JVM 提交请求,可能被忽略(比如 -XX:+DisableExplicitGC);即使执行,STW 停顿也会伤性能。教学观察除外。正确做法:设置堆大小上限(-Xmx),让 JVM 自己按水位线回收,并用工具观察。

§5.4 四种引用:从强到弱

引用类型	回收条件	典型用途
强引用(默认)	永不回收,直到不可达	日常所有 new
软引用 SoftReference	内存不足时才回收	图片/数据缓存(内存敏感)
弱引用 WeakReference	下一次 GC 必回收	缓存、观察者、WeakHashMap
虚引用 PhantomReference	对象回收后收到通知	直接内存回收、对象消亡跟踪(高级)

示例 5-4 WeakReference:让缓存"漏得掉"

```

import java.lang.ref.WeakReference;
import java.util.WeakHashMap;

public class WeakRefDemo {
    public static void main(String[] args) throws Exception {
        // ---- 弱引用:GC 后引用自动变 null ----
        BigObject obj = new BigObject();
        WeakReference<BigObject> ref = new WeakReference<>(obj);
        System.out.println("GC 前 get(): " + (ref.get() != null));    // true

        obj = null;          // 唯一强引用断开,对象只剩弱引用
        System.gc();
        Thread.sleep(100);
        System.out.println("GC 后 get(): " + (ref.get() != null));    // false

        // ---- WeakHashMap:键是弱引用,键被回收时条目自动消失 ----
        WeakHashMap<String, String> cache = new WeakHashMap<>();
        String key = new String("config");    // 不要用字面量,否则常量池强引用它
        cache.put(key, "v1");
        key = null;          // 键的强引用断开
        System.gc();
        Thread.sleep(100);
        System.out.println("缓存条目数: " + cache.size());          // 0
    }

    static class BigObject {
        byte[] payload = new byte[1024 * 1024];
    }
}

```

★ 重点:缓存的正确姿势

缓存是弱引用的经典场景:缓存数据的生命周期应该"跟着使用它的业务对象走"。强引用缓存 = 静态持有 = 永不回收 = 内存泄漏(见 §5.5);WeakHashMap 让条目在"业务键"消失后自动蒸发。生产级别的缓存框架(Caffeine、Redis)有更复杂的淘汰策略,但原理起点就是这里。

§5.5 内存泄漏四大源头

5.5.1 Java 内存泄漏 = 忘了放手

对象明明没用了,但代码里还留着到它的引用链,GC 认为它可达,永远不回收。四大高频源头:

1. **静态集合无限增长**: `static List<Session> sessions` 只 add 不 remove,或"全局缓存无上限"
2. **长生命周期对象持有短生命周期对象**:全局单例持有请求对象、监听器注册后从不注销
3. **资源未关闭**:文件/连接/流没 close,底层 native 资源泄漏(栈内存也涨)
4. **ThreadLocal 不清除**:线程池里的线程长期存活,ThreadLocal 值被线程强引用,形成泄漏(第9章)

示例 5-5 泄漏演示与修复

```

import java.util.ArrayList;
import java.util.List;

public class LeakDemo {
    // ---- 泄漏版:静态集合只进不出 ----
    static final List<byte[]> CACHE = new ArrayList<>();

    static void leak(int n) {
        for (int i = 0; i < n; i++) {
            CACHE.add(new byte[1024 * 1024]); // 1MB x n, 永远可达!
        }
    }

    // ---- 修复版:限长缓存,超出丢弃最旧 ----
    static void boundedCache(byte[] data) {
        if (CACHE.size() >= 10) {
            CACHE.remove(0); // 超过上限,移除最旧
        }
        CACHE.add(data);
    }

    public static void main(String[] args) throws Exception {
        // 模拟业务:每次请求缓存一个快照,快照用完应丢弃
        for (int req = 0; req < 1000; req++) {
            byte[] snapshot = new byte[1024 * 1024];
            // leak(1) 会累积 1000MB;boundedCache 只留 10MB
            boundedCache(snapshot);
            snapshot = null; // 用完就放手
        }
        System.gc();
        Thread.sleep(200);
        long used = (Runtime.getRuntime().totalMemory()
            - Runtime.getRuntime().freeMemory()) / 1024 / 1024;
        System.out.println("处理后已用内存(约): " + used + " MB");
        System.out.println("缓存条目: " + CACHE.size()); // 10
    }
}

```

⚠ 易错点 3:把对象赋 null 不是好习惯

看到"C 程序员式清理" `obj = null` ——多数时候没必要:局部变量随栈帧弹出自然消失。但 **长生命周期容器里"用完了"的对象**,显式置 `null` 或 `remove` 是有意义的(帮助 GC 尽早回收)。判断标准:这个引用会不会活得比对象久?会 → 放手;不会 → 别画蛇添足。

§5.6 try-with-resources:Java 的 RAII

5.6.1 语法与语义

try-with-resources:在 `try` 括号里声明资源,代码块结束时(无论正常、`return`、异常) **自动按声明逆序调用 `close()`**。资源类必须实现 `AutoCloseable` (其子接口 `Closeable` 的 `close` 不抛普通异常)。对照 C:作用域结束自动调用析构函数——语义等价,语法更显式。

示例 5-6 自定义资源 + try-with-resources

```

import java.io.BufferedReader;
import java.io.FileReader;

public class TwrDemo {
    // 自定义资源:实现 AutoCloseable 就能进 try-with-resources
    static class LogFile implements AutoCloseable {
        private final String name;

        LogFile(String name) { this.name = name; System.out.println("打开 " + name); }

        void write(String line) { System.out.println(" [写入] " + name + ": " + line); }

        @Override
        public void close() { // 必须实现;声明 throws Exception 也行
            System.out.println("关闭 " + name); // 一定被执行
        }
    }

    public static void main(String[] args) throws Exception {
        // 多个资源:用分号分隔,按声明逆序关闭(先 b 后 a)
        try (LogFile a = new LogFile("a.log");
            LogFile b = new LogFile("b.log")) {
            a.write("hello");
            b.write("world");
        } // 出块自动关闭,连 catch/finally 都不需要

        // 标准库示例:读文件,不用手写 finally { in.close(); }
        try (BufferedReader in = new BufferedReader(
            new FileReader("data.txt"))) {
            String line = in.readLine();
            System.out.println("首行: " + line);
        } // 文件句柄在这里被自动关闭;文件不存在时 catch 一下即可
    }
}

```



工程建议:资源处理三板斧

1. 凡是"打开就要关"的东西(流、连接、通道、锁),一律 try-with-resources ,不要手写 try/finally/close 三板斧——代码更短,且异常路径也不会漏关
2. 多资源按"先开的后关"写,括号里用分号分隔,别嵌套 try
3. close() 失败(比如网络断开)会让真正的业务异常被吞——需要时用 try {...} catch (Exception e) { e.addSuppressed(...) } 语义,JDK 已自动处理 suppressed 异常(可用 getSuppressed 查看)

§5.7 OOM 诊断与 JVM 参数

5.7.1 常用 JVM 启动参数

参数	含义	生产建议
-Xms256m	堆初始大小	与 -Xmx 相同,避免运行时扩容抖动
-Xmx1g	堆最大大小	必须设置上限,防止 OOM 拖垮整机
-Xss512k	单线程栈大小	递归深再调;默认约 512k-1m
-XX:+HeapDumpOnOutOfMemoryError	OOM 时自动导出堆转储	生产必开,配合 -XX:HeapDumpPath 指定路径
-XX:MaxMetaspaceSize	元空间上限	类加载过多(动态生成类)时保护

示例 5-7 生产启动参数示例(命令行)


```
# 生产启动示例:堆固定 512m,防止扩容抖动;OOM 自动导出堆转储
java -Xms512m -Xmx512m -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/logs/dumps/ -jar app.jar

# 本地调试:调大栈空间,观察递归深度变化
java -Xss2m StackDepthDemo

# 查看 GC 频率(每秒一次):jstat 是定位 GC 风暴的第一把刀
jstat -gcutil 12345 1000
```

5.7.2 诊断工具清单

- **jps**:列出 JVM 进程;**jstat -gcutil <pid>**:每秒看 GC 频率与堆占用——先看这个,判断是不是 GC 风暴
- **jmap -dump**:手动导出堆转储(生产慎用,会 STW);OOM 自动转储后用 **Eclipse MAT / VisualVM** 分析"支配树",找谁占着内存、被谁引用
- **jcmd**:综合诊断命令, **jcmd <pid> GC.heap_info** 等

⚠ 易错点 4:OOM 处理流程别反

正确顺序:① 先确认 OOM 类型(heap?metaspace?thread?);② 用 jstat 看 GC 频率;③ 导出堆转储分析支配树找"大对象+引用链";④ 改代码(防泄漏/限长/分页),而不是盲目加 -Xmx。90% 的 OOM 是代码问题,10% 才是堆太小。

本章速查表

主题	要点
五大区域	程序计数器 / 虚拟机栈 / 本地方法栈 / 堆 / 元空间(JDK8+ 取代方法区)
栈 vs 堆	局部变量在栈帧;所有 new 的对象在堆;栈帧弹出→引用消失→对象可回收
逃逸	对象被返回/存入字段/集合 = 生命周期超过所在方法 = 逃逸
GC 原理	从 GC Roots 做可达性分析;不可达即回收;循环引用自动解决
分代	新生代(Eden+Survivor,频繁 Minor GC)→ 老年代(Major GC 慢);STW 暂停业务线程
System.gc()	只是建议,生产不要调;教学观察除外
四种引用	强(默认)/ 软(内存不足才回收,缓存)/ 弱(下次 GC 必回收)/ 虚(回收通知)
WeakHashMap	键用弱引用;键无强引用时条目自动消失;缓存首选
泄漏四源	静态集合只增不减 / 长生命周期持短生命 / 资源未关 / ThreadLocal 不清
finalize	已废弃,生产禁用;资源释放在 try-with-resources
try-with-resources	try (资源) { };自动 close,逆序关闭;资源须实现 AutoCloseable
JVM 参数	-Xmx 设上限;-XX:+HeapDumpOnOutOfMemoryError 必开;-Xss 管递归深度
诊断流程	jps → jstat -gcutil → 堆转储 + MAT 支配树 → 改代码,别盲目加堆
OOM 与栈溢出	OutOfMemoryError 是堆耗尽;StackOverflowError 是递归过深;都是 Error

最小必会清单

- 能画出 JVM 五大内存区域并说出各自存什么、对应 C 的什么
- 能解释"栈上的引用、堆上的对象"与对象逃逸
- 能解释可达性分析与分代收集,说出 STW 的含义
- 能说出四种引用的回收条件,并写出 WeakHashMap 缓存
- 能说出 Java 内存泄漏的定义与四大源头,并写出修复
- 能写出 try-with-resources 并解释它与 C RAII 的关系

- 能说出 OOM 诊断四步流程

自测题

Q 1. 递归调用 10 万层会发生什么?怎么解决?

✓ 参考答案

StackOverflowError(不是 OOM):每层方法调用压一个栈帧,默认栈大小约 512KB-1MB,装不下 10 万帧。解决:改迭代(尾递归 Java 不优化)、加深思量后调 -Xss,或根本不该用递归(深度 $O(n)$ 的算法)。

Q 2. 判断:C 的内存泄漏是"忘了释放",Java 的内存泄漏是"忘了放手"。请解释"放手"指什么。

✓ 参考答案

C 的泄漏:malloc 后没有 free,内存永远占着。Java 的泄漏:对象仍然可达(被静态集合、长生命周期对象、线程持有),GC 永远不会回收它。所以"放手"= 断开不再需要的引用(remove 出集合、置 null、关闭资源)。

Q 3. 一个 WeakHashMap 缓存,键是 String,值为配置数据。为什么代码里 `cache.put("key", v)` 用字面量键,条目永远不消失?

✓ 参考答案

字符串字面量被常量池/元空间强引用,它的强引用链不断,弱引用键永远不会变 null,条目也就不会被清除。想演示弱引用行为,必须用 `new String("key")` 并断开局部变量——这也解释了为什么"缓存键尽量用业务对象"。

Q 4. 下面的代码有什么问题?`void read() throws Exception { FileInputStream in = new FileInputStream("a.txt"); int b = in.read(); System.out.println(b); }`

✓ 参考答案

文件句柄泄漏:read 抛异常或方法返回时 in 没关闭,底层 native 句柄泄漏。修复:`try (FileInputStream in = new FileInputStream("a.txt")) { ... }`,自动关闭,异常路径也不会漏。

Q 5. 生产环境出现 OOM:java.lang.OutOfMemoryError: Java heap space。给出排查步骤。

✓ 参考答案

① 确认是堆 OOM(而非 metaspace/线程);② jstat -gcutil 观察 GC 频率(是否 GC 风暴);③ 用 -XX:+HeapDumpOnOutOfMemoryError 生成的堆转储,在 MAT 里看支配树找"最大的对象+引用链";④ 修复代码(静态集合限长/释放资源/分批处理),必要时才调 -Xmx。不要一上来就加内存。

Q 6. 软引用与弱引用的区别?分别适合什么场景?

✓ 参考答案

软引用:内存不足时才回收,适合"丢了可惜但可重建"的缓存(大图片、计算结果);弱引用:下一次 GC 就回收,适合"键跟着业务对象走"的缓存与观察者注册。生产缓存框架(如 Caffeine)内部会综合两者做淘汰策略。

章末实验题:内存观察与资源管理

实验 5:内存观察与资源管理综合实验

实验目标:编写一个程序,综合运用本章全部知识点:内存区域观察、对象生命周期、GC 观察、弱引用缓存、内存泄漏修复、try-with-resources 资源管理。

实验要求:

- 任务 1:用 Runtime 打印 JVM 总内存与初始已用内存(覆盖 §5.1-5.2 堆与栈)
- 任务 2:创建大量短命大对象并打印每轮后的已用内存,观察 GC 回收(覆盖 §5.3 分代收集)
- 任务 3:实现一个 WeakReference 缓存,演示"业务键消失后缓存条目自动消失"(覆盖 §5.4 弱引用)
- 任务 4:实现一个静态 List 缓存并故意只增不减,随后加上限长修复,对比两者内存占用(覆盖 §5.5 泄漏与修复)
- 任务 5:自定义 AutoCloseable 资源类,用 try-with-resources 管理两个资源,观察关闭顺序(覆盖 §5.6)
- 任务 6:用递归制造 StackOverflowError,捕获后打印最大深度(覆盖 §5.7 栈区)
- 任务 7:程序运行结束后打印最终内存占用,输出完整观察报告(覆盖 §5.7 诊断)

实验环境:JDK 17+;建议 `java -Xmx256m MemoryLab` 运行,体会小堆下的 GC 压力。

第一步 写 main 骨架与任务 1、2(内存打印),跑通后再逐个实现 3-6

第二步 务 4 先跑泄漏版,用内存数字"眼见为实",再改限长版对比

第三步 务 5 观察关闭顺序:声明 a、b 两个资源,应输出"先关 b 后关 a"

参考答案要点

```

import java.lang.ref.WeakReference;
import java.util.ArrayList;
import java.util.List;

/** 内存观察与资源管理 — 第5章综合实验参考答案
 * 覆盖:堆/栈观察、GC 回收、弱引用缓存、泄漏修复、try-with-resources、栈溢出
 */
public class MemoryLab {

    // 任务 4:静态缓存(泄漏版)
    static final List<byte[]> CACHE = new ArrayList<>();
    static final int CACHE_LIMIT = 10;

    // 任务 5:自定义资源
    static class LogFile implements AutoCloseable {
        private final String name;

        LogFile(String name) {
            this.name = name;
            System.out.println(" 打开资源: " + name);
        }

        void write(String line) {
            System.out.println("    写入: " + name + " / " + line);
        }

        @Override
        public void close() {
            System.out.println(" 关闭资源: " + name);
        }
    }

    static long usedMb() {
        Runtime rt = Runtime.getRuntime();
        return (rt.totalMemory() - rt.freeMemory()) / 1024 / 1024;
    }

    // 任务 4 泄漏版:只进不出
    static void leakAdd(byte[] data) {
        CACHE.add(data);
    }

    // 任务 4 修复版:限长,超出丢最旧
    static void safeAdd(byte[] data) {
        if (CACHE.size() >= CACHE_LIMIT) {
            CACHE.remove(0);
        }
        CACHE.add(data);
    }

    // 任务 6:栈溢出
    static int depth = 0;

    static void dive() {
        depth++;
        dive();
    }

    public static void main(String[] args) throws Exception {
        System.out.println("== 任务 1:内存地图 ==");
        Runtime rt = Runtime.getRuntime();
        System.out.println(" 堆总内存(约): " + rt.totalMemory() / 1024 / 1024 + " MB");
        System.out.println(" 初始已用(约): " + usedMb() + " MB");

        System.out.println("== 任务 2:GC 回收观察 ==");
        for (int round = 1; round <= 3; round++) {
            for (int i = 0; i < 500; i++) {
                new byte[1024 * 256]; // 每轮制造约 128MB 垃圾
            }
        }
    }
}

```

```

    }
    System.gc(); // 教学用途;生产禁止
    Thread.sleep(100);
    System.out.println(" 第 " + round + " 轮后已用(约): " + usedMb() + " MB");
}

System.out.println("== 任务 3:弱引用缓存 ==");
byte[] payload = new byte[1024 * 512];
WeakReference<byte[]> ref = new WeakReference<>(payload);
System.out.println(" 断开强引用前: " + (ref.get() != null));
payload = null;
System.gc();
Thread.sleep(100);
System.out.println(" 断开强引用后: " + (ref.get() != null) + "(false=已被回收)");

System.out.println("== 任务 4:泄漏 vs 限长 ==");
long before = usedMb();
for (int i = 0; i < 30; i++) {
    safeAdd(new byte[1024 * 1024]); // 限长版:始终只有 10 条
}
System.gc();
Thread.sleep(100);
System.out.println(" 限长版缓存条目: " + CACHE.size() + ", 已用(约): " + usedMb() + " MB");
// 若改为 leakAdd:条目 30,已用内存显著增长—泄漏肉眼可见
CACHE.clear();
System.gc();
Thread.sleep(100);
System.out.println(" 清空后已用(约): " + usedMb() + " MB, 之前: " + before + " MB");

System.out.println("== 任务 5:try-with-resources ==");
try (LogFile a = new LogFile("a.log");
     LogFile b = new LogFile("b.log")) {
    a.write("hello");
    b.write("world");
} // 观察输出顺序:先关 b,后关 a(逆序关闭)

System.out.println("== 任务 6:栈溢出 ==");
try {
    dive();
} catch (StackOverflowError e) {
    System.out.println(" 递归最大深度约: " + depth);
}

System.out.println("== 任务 7:最终报告 ==");
System.out.println(" 结束已用(约): " + usedMb() + " MB");
System.out.println(" 实验完成:全部任务输出正常。");
}
}

```

设计说明:任务 2 用“每轮制造垃圾后内存不累积”直观证明 GC 在工作;任务 4 的 safeAdd 与 leakAdd 只需换一行即可对比泄漏;任务 5 输出“先关 b 后关 a”验证逆序关闭。扩展方向:任务 4 改成 WeakHashMap 缓存(与任务 3 呼应);用 -XX:+HeapDumpOnOutOfMemoryError 跑一个真正 OOM 的变体(把 safeAdd 改成无上限)并用 MAT 分析。

下一章预告:第6章 面向对象。类与对象、封装继承多态、equals/hashCode 契约、内部类与匿名类——内存管理教你怎么活,这一章教你用什么形状活。